

	Linear Data structure	Non-Linear Data structure
Basic	In this structure, the elements are arranged sequentially or linearly and attached to one another.	In this structure, the elements are arranged hierarchically or non-linear manner.
Types	Arrays, linked list, stack, queue are the types of a linear data structure.	Trees and graphs are the types of a non-linear data structure.
implementation	Due to the linear organization, they are easy to implement.	Due to the non-linear organization, they are difficult to implement.
Traversal	As linear data structure is a single level, so it requires a single run to traverse each data item.	The data items in a non-linear data structure cannot be accessed in a single run. It requires multiple runs to be traversed.
Arrangement	Each data item is attached to the previous and next items.	Each item is attached to many other items.
Levels	This data structure does not contain any hierarchy, and all the data elements are organized in a single	In this, the data elements are arranged in multiple levels.

Levels	This data structure does not contain any hierarchy, and all the data elements are organized in a single level.	In this, the data elements are arranged in multiple levels.
Memory utilization	In this, the memory utilization is not efficient.	In this, memory is utilized in a very efficient manner.
Time complexity	The time complexity of linear data structure increases with the increase in the input size.	The time complexity of non-linear data structure often remains same with the increase in the input size.
Applications	Linear data structures are mainly used for developing the software.	Non-linear data structures are used in image processing and Artificial Intelligence .

Array	Linked list
An array is a collection of elements of a similar data type.	A linked list is a collection of objects known as a node where node consists of two parts, i.e., data and address.
Array elements store in a contiguous memory location.	Linked list elements can be stored anywhere in the memory or randomly stored.
Array works with a static memory. Here static memory means that the memory size is fixed and cannot be changed at the run time.	The Linked list works with dynamic memory. Here, dynamic memory means that the memory size can be changed at the run time according to our requirements.
Array elements are independent of each other.	Linked list elements are dependent on each other. As each node contains the address of the next node so to access the next node, we need to access its previous node.
Array takes more time while performing any operation like insertion, deletion, etc.	Linked list takes less time while performing any operation like insertion, deletion, etc.
Accessing any element in an array is faster as the element in an array can be directly accessed	Accessing an element in a linked list is slower as it starts traversing from the first element of the

	node so to access the next node, we need to access its previous node.
Array takes more time while performing any operation like insertion, deletion, etc.	Linked list takes less time while performing any operation like insertion, deletion, etc.
Accessing any element in an array is faster as the element in an array can be directly accessed through the index.	Accessing an element in a linked list is slower as it starts traversing from the first element of the linked list.
In the case of an array, memory is allocated at compile-time.	In the case of a linked list, memory is allocated at run time.
Memory utilization is inefficient in the array. For example, if the size of the array is 6, and array consists of 3 elements only then the rest of the space will be unused.	Memory utilization is efficient in the case of a linked list as the memory can be allocated or deallocated at the run time according to our requirement.

Basis for comparison	Stack	Queue
Principle	It follows the principle LIFO (Last In- First Out), which implies that the element which is inserted last would be the first one to be deleted.	It follows the principle FIFO (First In -First Out), which implies that the element which is added first would be the first element to be removed from the list.
Structure	It has only one end from which both the insertion and deletion take place, and that end is known as a top.	It has two ends, i.e., front and rear end. The front end is used for the deletion while the rear end is used for the insertion.
Number of pointers used	It contains only one pointer known as a top pointer. The top pointer holds the address of the last inserted or the topmost element of the stack.	It contains two pointers front and rear pointer. The front pointer holds the address of the first element, whereas the rear pointer holds the address of the last element in a queue.
Operations performed	It performs two operations, push and pop. The push operation inserts the element in a list while	It performs mainly two operations, enqueue and dequeue. The enqueue operation performs the insertion of the

Operations performed	It performs two operations, push and pop. The push operation inserts the element in a list while the pop operation removes the element from the list.	It performs mainly two operations, enqueue and dequeue. The enqueue operation performs the insertion of the elements in a queue while the dequeue operation performs the deletion of the elements from the queue.
Examination of the empty condition	If $\text{top} == -1$, which means that the stack is empty.	If $\text{front} == -1$ or $\text{front} = \text{rear} + 1$, which means that the queue is empty.
Examination of full condition	If $\text{top} == \text{max} - 1$, this condition implies that the stack is full.	If $\text{rear} == \text{max} - 1$, this condition implies that the stack is full.
Variants	It does not have any types.	It is of three types like priority queue, circular queue and double ended queue.
Implementation	It has a simpler implementation.	It has a comparatively complex implementation than a stack.
Visualization	A Stack is visualized as a vertical collection.	A Queue is visualized as a horizontal collection.

Basis of comparison	Linear Queue	Circular Queue
Meaning	The linear queue is a type of linear data structure that contains the elements in a sequential manner.	The circular queue is also a linear data structure in which the last element of the Queue is connected to the first element, thus creating a circle.
Insertion and Deletion	In linear queue, insertion is done from the rear end, and deletion is done from the front end.	In circular queue, the insertion and deletion can take place from any end.
Memory space	The memory space occupied by the linear queue is more than the circular queue.	It requires less memory as compared to linear queue.
Memory utilization	The usage of memory is inefficient.	The memory can be more efficiently utilized.
Order of execution	It follows the FIFO principle in order to perform the tasks.	It has no specific order for execution.

Basis of comparison	Linear search	Binary search
Definition	The linear search starts searching from the first element and compares each element with a searched element till the element is not found.	It finds the position of the searched element by finding the middle element of the array.
Sorted data	In a linear search, the elements don't need to be arranged in sorted order.	The pre-condition for the binary search is that the elements must be arranged in a sorted order.
Implementation	The linear search can be implemented on any linear data structure such as an array, linked list, etc.	The implementation of binary search is limited as it can be implemented only on those data structures that have two-way traversal.
Approach	It is based on the sequential approach.	It is based on the divide and conquer approach.
Size	It is preferable for the small-sized data sets.	It is preferable for the large-size data sets.

Approach	It is based on the sequential approach.	It is based on the divide and conquer approach.
Size	It is preferable for the small-sized data sets.	It is preferable for the large-size data sets.
Efficiency	It is less efficient in the case of large-size data sets.	It is more efficient in the case of large-size data sets.
Worst-case scenario	In a linear search, the worst-case scenario for finding the element is $O(n)$.	In a binary search, the worst-case scenario for finding the element is $O(\log_2 n)$.
Best-case scenario	In a linear search, the best-case scenario for finding the first element in the list is $O(1)$.	In a binary search, the best-case scenario for finding the first element in the list is $O(1)$.
Dimensional array	It can be implemented on both a single and multidimensional array.	It can be implemented only on a multidimensional array.

Basis of comparison	Singly linked list	Doubly linked list
Definition	A single linked list is a list of nodes in which node has two parts, the first part is the data part, and the next part is the pointer pointing to the next node in the sequence of nodes.	A doubly linked list is also a collection of nodes in which node has three fields, the first field is the pointer containing the address of the previous node, the second is the data field, and the third is the pointer containing the address of the next node.
Access	The singly linked list can be traversed only in the forward direction.	The doubly linked list can be accessed in both directions.
List pointer	It requires only one list pointer variable, i.e., the head pointer pointing to the first node.	It requires two list pointer variables, head and last . The head pointer points to the first node, and the last pointer points to the last node of the list.
Memory space	It utilizes less memory space.	It utilizes more memory space.
Efficiency	It is less efficient as compared to a doubly-linked list.	It is more efficient.

List pointer	It requires only one list pointer variable, i.e., the head pointer pointing to the first node.	It requires two list pointer variables, head and last . The head pointer points to the first node, and the last pointer points to the last node of the list.
Memory space	It utilizes less memory space.	It utilizes more memory space.
Efficiency	It is less efficient as compared to a doubly-linked list.	It is more efficient.
Implementation	It can be implemented on the stack.	It can be implemented on stack, heap and binary tree.
Complexity	In a singly linked list, the time complexity for inserting and deleting an element from the list is $O(n)$.	In a doubly-linked list, the time complexity for inserting and deleting an element is $O(1)$.

Basis for comparison	Binary tree	Binary search tree
Definition	A binary tree is a non-linear data structure in which a node can have utmost two children, i.e., a node can have 0, 1 or maximum two children. A binary search tree is an ordered binary tree in which some order is followed to organize the nodes in a tree.	
Structure	The structure of the binary tree is that the first node or the topmost node is known as the root node. Each node in a binary tree contains the left pointer and the right pointer. The left pointer contains the address of the left subtree, whereas right pointer contains the address of right subtree.	The binary search tree is one of the types of binary tree that has the value of all the nodes in the left subtree lesser or equal to the root node, and the value of all the nodes in a right subtree are greater than or equal to the value of the root node.
Operations	The operations that can be implemented on a binary tree are insertion, deletion, and traversal.	Binary search trees are the sorted binary trees that provide fast insertion, deletion and search. Lookups mainly implement binary search as all the keys are arranged in

Structure	The structure of the binary tree is that the first node or the topmost node is known as the root node. Each node in a binary tree contains the left pointer and the right pointer. The left pointer contains the address of the left subtree, whereas right pointer contains the address of right subtree.	The binary search tree is one of the types of binary tree that has the value of all the nodes in the left subtree lesser or equal to the root node, and the value of all the nodes in a right subtree are greater than or equal to the value of the root node.
Operations	The operations that can be implemented on a binary tree are insertion, deletion, and traversal.	Binary search trees are the sorted binary trees that provide fast insertion, deletion and search. Lookups mainly implement binary search as all the keys are arranged in sorted order.
types	Four types of binary trees are Full Binary Tree, Complete Binary Tree, Perfect Binary Tree, and Extended Binary Tree.	There are different types of binary search trees such as AVL trees, Splay tree, Tango trees, etc.

Basis for comparison	Tree	Graph
Definition	Tree is a non-linear data structure in which elements are arranged in multiple levels.	A Graph is also a non-linear data structure.
Structure	It is a collection of edges and nodes. For example, node is represented by N and edge is represented as E, so it can be written as: $T = \{N, E\}$	It is a collection of vertices and edges. For example, vertices are represented by V, and edge is represented as 'E', so it can be written as: $T = \{V, E\}$
Root node	In tree data structure, there is a unique node known as a parent node. It represents the topmost node in the tree data structure.	In graph data structure, there is no unique node.
Loop formation	It does not create any loop or cycle.	In graph, loop or cycle can be formed.
Model type	It is a hierarchical model because nodes are arranged in multiple level, and that creates hierarchy. For example, any organization	It is a network model. For example, facebook is a social network that uses the graph data structure

Loop formation	It does not create any loop or cycle.	In graph, loop or cycle can be formed.
Model type	It is a hierarchical model because nodes are arranged in multiple level, and that creates a hierarchy. For example, any organization will have a hierarchical model.	It is a network model. For example, facebook is a social network that uses the graph data structure.
Edges	If there are n nodes then there would be n-1 number of edges.	The number of edges depends on the graph.
Type of edge	Tree data structure will always have directed edges.	In graph data structure, all the edges can either be directed edges, undirected edges, or both.
Applications	It is used for inserting, deleting or searching any element in tree.	It is mainly used for finding the shortest path in the network.

Binary Search tree	AVL tree
Every binary search tree is a binary tree because both the trees contain the utmost two children.	Every AVL tree is also a binary tree because AVL tree also has the utmost two children.
In BST, there is no term exists, such as balance factor.	In the AVL tree, each node contains a balance factor, and the value of the balance factor must be either -1, 0, or 1.
Every Binary Search tree is not an AVL tree because BST could be either a balanced or an unbalanced tree.	Every AVL tree is a binary search tree because the AVL tree follows the property of the BST.
Each node in the Binary Search tree consists of three fields, i.e., left subtree, node value, and the right subtree.	Each node in the AVL tree consists of four fields, i.e., left subtree, node value, right subtree, and the balance factor.
In the case of Binary Search tree, if we want to insert any node in the tree then we compare the node value with the root value; if the value of node is greater than the root node value then the node is inserted to	In the case of AVL tree, first, we will find the suitable place to insert the node. Once the node is inserted, we will calculate the balance factor of each node. If the balance factor of

In the case of Binary Search tree, if we want to insert any node in the tree then we compare the node value with the root value; if the value of node is greater than the root node value then the node is inserted to the right subtree otherwise the node is inserted to the left subtree. Once the node is inserted, there is no need of checking the height balance factor for the insertion to be completed.	In the case of AVL tree, first, we will find the suitable place to insert the node. Once the node is inserted, we will calculate the balance factor of each node. If the balance factor of each node is satisfied, the insertion is completed. If the balance factor is greater than 1, then we need to perform some rotations to balance the tree.
In Binary Search tree, the height or depth of the tree is $O(n)$ where n is the number of nodes in the Binary Search tree.	In AVL tree, the height or depth of the tree is $O(\log n)$.
It is simple to implement as we have to follow the Binary Search properties to insert the node.	It is complex to implement because in AVL tree, we have to first construct the AVL tree, and then we need to check height balance. If the height is imbalance then we need to perform some rotations to balance the tree.
BST is not a balanced tree because it does not follow the concept of the balance factor.	AVL tree is a height balanced tree because it follows the concept of the balance factor.

In Binary Search tree, the height or depth of the tree is $O(n)$ where n is the number of nodes in the Binary Search tree.	In AVL tree, the height or depth of the tree is $O(\log n)$.
It is simple to implement as we have to follow the Binary Search properties to insert the node.	It is complex to implement because in AVL tree, we have to first construct the AVL tree, and then we need to check height balance. If the height is imbalance then we need to perform some rotations to balance the tree.
BST is not a balanced tree because it does not follow the concept of the balance factor.	AVL tree is a height balanced tree because it follows the concept of the balance factor.
Searching is inefficient in BST when there are large number of nodes available in the tree because the height is not balanced.	Searching is efficient in AVL tree even when there are large number of nodes in the tree because the height is balanced.

Parameter	Red Black Tree	AVL Tree
Searching	Red Black tree does not provide efficient searching as Red Black Trees are roughly balanced.	AVL trees provide efficient searching as it is strictly balanced tree.
Insertion and Deletion	Insertion and Deletion are easier in Red Black tree as it requires fewer rotations to balance the tree.	Insertion and Deletion are complex in AVL tree as it requires multiple rotations to balance the tree.
Color of the node	In the Red-Black tree, the color of the node is either Red or Black.	In the case of AVL trees, there is no color of the node.
Balance factor	It does not contain any balance factor. It stores only one bit of information that denotes either Red or Black color of the node.	Each node has a balance factor in AVL tree whose value can be 1, 0, or -1. It requires extra space to store the balance factor per node.
Strictly balanced	Red-black trees are not strictly balanced.	AVL trees are strictly balanced, i.e., the left subtree's height and the height of the right subtree differ by at most 1.

B tree	B+ tree
In the B tree, all the keys and records are stored in both internal as well as leaf nodes.	In the B+ tree, keys are the indexes stored in the internal nodes and records are stored in the leaf nodes.
In B tree, keys cannot be repeatedly stored, which means that there is no duplication of keys or records.	In the B+ tree, there can be redundancy in the occurrence of the keys. In this case, the records are stored in the leaf nodes, whereas the keys are stored in the internal nodes, so redundant keys can be present in the internal nodes.
In the Btree, leaf nodes are not linked to each other.	In B+ tree, the leaf nodes are linked to each other to provide the sequential access.
In Btree, searching is not very efficient because the records are either stored in leaf or internal nodes.	In B+ tree, searching is very efficient or quicker because all the records are stored in the leaf nodes.
Deletion of internal nodes is very slow and a time-consuming process as we need to consider the child of the deleted key also.	Deletion in B+ tree is very fast because all the records are stored in the leaf nodes so we do not have to consider the child of the node.
In Btree, sequential access is not	In the B+ tree, all the leaf nodes are connected to each

In Btree, sequential access is not possible.	In the B+ tree, all the leaf nodes are connected to each other through a pointer, so sequential access is possible.
In Btree, the more number of splitting operations are performed due to which height increases compared to width,	B+ tree has more width as compared to height.
In Btree, each node has atleast two branches and each node contains some records, so we do not need to traverse till the leaf nodes to get the data.	In B+ tree, internal nodes contain only pointers and leaf nodes contain records. All the leaf nodes are at the same level, so we need to traverse till the leaf nodes to get the data.
The root node contains atleast 2 to m children where m is the order of the tree.	The root node contains atleast 2 to m children where m is the order of the tree.

Parameter	Quick Sort	Merge Sort
Definition	It is a quick sort algorithm that arranges the given elements into ascending order by comparing and interchanging the position of the elements.	It is a merge sort algorithm that arranges the given sets of elements in ascending order using the divide and conquer technique, and then compare with corresponding elements to sort the array.
Principle	It works on divide and conquer techniques.	It works on divide and conquer techniques.
Partition of elements	In quick sort, the array can be divide into any ratio.	Merge sort partition an array into two sub array ($N/2$).
Efficiency	It is more efficient and work faster in smaller size array, as compared to the merge sort.	It is more efficient and work faster in larger data sets or array, as compare to the quick sort.
Sorting method	It is an internal sorting method that sort the array or data available on main memory.	It is an external sorting method that sort the array or data sets available on external file.
Time	Its worst time complexity is O	Whereas, it's worst time complexity is O

5	Sorting method	It is an internal sorting method that sort the array or data available on main memory.	It is an external sorting method that sort the array or data sets available on external file.
6	Time complexity	Its worst time complexity is $O(n^2)$.	Whereas, it's worst time complexity is $O(n \log n)$.
7	Preferred	It is a sorting algorithm that is applicable for large unsorted arrays.	Whereas, the merge sort algorithm that is preferred to sort the linked lists.
8	Stability	Quick sort is an unstable sort algorithm. But we can made it stable by using some changes in programming code.	Merge sort is a stable sort algorithm that contains two equal elements with same values in sorted output.
9	Requires Space	It does not require any additional space to perform the quick sort.	It requires the additional space as temporary array to merge two sub arrays.
10.	Functionality	Compare each element with the pivot until all elements are arranged in ascending order.	Whereas, the merge sort splits the array into two parts ($N/2$) and it continuously divides the array until an element is left.

	BFS	DFS
Full form	BFS stands for Breadth First Search.	DFS stands for Depth First Search.
Technique	It a vertex-based technique to find the shortest path in a graph.	It is an edge-based technique because the vertices along the edge are explored first from the starting to the end node.
Definition	BFS is a traversal technique in which all the nodes of the same level are explored first, and then we move to the next level.	DFS is also a traversal technique in which traversal is started from the root node and explore the nodes as far as possible until we reach the node that has no unvisited adjacent nodes.
Data Structure	Queue data structure is used for the BFS traversal.	Stack data structure is used for the BFS traversal.
Backtracking	BFS does not use the backtracking concept.	DFS uses backtracking to traverse all the unvisited nodes.
Number of edges	BFS finds the shortest path having a minimum number of edges to	In DFS, a greater number of edges are required to traverse from the source vertex to

Number of edges	BFS finds the shortest path having a minimum number of edges to traverse from the source to the destination vertex.	In DFS, a greater number of edges are required to traverse from the source vertex to the destination vertex.
Optimality	BFS traversal is optimal for those vertices which are to be searched closer to the source vertex.	DFS traversal is optimal for those graphs in which solutions are away from the source vertex.
Speed	BFS is slower than DFS.	DFS is faster than BFS.
Suitability for decision tree	It is not suitable for the decision tree because it requires exploring all the neighboring nodes first.	It is suitable for the decision tree. Based on the decision, it explores all the paths. When the goal is found, it stops its traversal.
Memory efficient	It is not memory efficient as it requires more memory than DFS.	It is memory efficient as it requires less memory than BFS.

Stack	Heap
Stack provides static memory allocation, i.e., it is used to store the temporary variables.	Heap provides dynamic memory allocation. By default, all the global variables are stored in the heap.
It is a linear data structure means that elements are stored in the linear manner, i.e., one data after another.	It is hierarchical data structure means that the elements are stored in the form of tree.
It is used to access the local variables.	It is used to access the global variables by default.
The size of the stack memory is limited which is dependent on the OS.	The size of the memory is not limited.
As it is a linear data structure, so data is stored in the contiguous blocks.	As it is hierarchical data structure, so elements are stored in the random manner.
In stack, the allocation and deallocation are automatically managed.	In heap, the memory is manually managed.
The implementation of stack can be done in three forms using array, linked list and dynamic memory.	The implementation of heap can be done in two forms using arrays and trees.
The main issue that occurs with a stack is the shortage	The main issue that occurs with a heap is

In stack, the allocation and deallocation are automatically managed.	In heap, the memory is manually managed.
The implementation of stack can be done in three forms using array, linked list and dynamic memory.	The implementation of heap can be done in two forms using arrays and trees.
The main issue that occurs with a stack is the shortage of memory because the memory size cannot be changed at the runtime. The size of the stack is determined at the compile time.	The main issue that occurs with a heap is the memory fragmentation. Here, memory fragmentation means that the memory gets wasted.
It is of fixed size.	It is flexible to use as the size of the heap can vary as per our needs.
The access time in stack is faster.	The access time in heap is slower.
The size of the stack memory is decided by the operating system.	The size of the heap memory is decided by the programmers.
The scope of the variable cannot be changed.	The scope of the variable can be changed.

Bubble sort	Selection sort
In bubble sort, two adjacent elements are compared. If the adjacent elements are not at the correct position, swapping would be performed.	In selection sort, the minimum element is selected from the array and swap with an element which is at the beginning of the unsorted sub array.
The time complexities in best case and worst case are $O(n)$ and $O(n^2)$ respectively.	The time complexity in both best and worst cases is $O(n^2)$.
It is not an efficient sorting technique.	It is an efficient sorting technique as compared to Bubble sort.
It uses an exchanging method.	It uses a selection method.
It is slower than the selection sort as a greater number of comparisons is required.	It is faster than the bubble sort as a lesser number of comparisons is required.

Array	Stack
It is a data structure that consists of a collection of elements that are identified by their indexes, where the first index is available at index 0.	It is an abstract data type that consists of a collection of elements that implements the LIFO (Last In First Out) principle.
It is a collection of elements of the same data type.	It is a collection of elements of different data types.
It provides random-access, i.e., read and write operations would be performed to any element at any position through their indexes.	As it implements LIFO so it has limited-access. We can access the only top element of the stack using push and pop operations.
It is rich in methods or operations like sorting, traversing, reverse, push, pop, etc.	The limited operations can be performed on a stack like push, pop, peek, etc.
It is a data type. It is an abstract data type.	
It is used when we know all the data to be processed and require constant changes at any element.	It is good to use when there are dynamic processes. It is useful when we do not know how much data would be required.

Primitive data structure	Non-primitive data structure
Primitive data structure is a kind of data structure that stores the data of only one type.	Non-primitive data structure is a type of data structure that can store the data of more than one type.
Examples of primitive data structure are integer, character, float.	Examples of non-primitive data structure are Array, Linked list, stack.
Primitive data structure will contain some value, i.e., it cannot be NULL.	Non-primitive data structure can consist of a NULL value.
The size depends on the type of the data structure.	In case of non-primitive data structure, size is not fixed.
It starts with a lowercase character.	It starts with an uppercase character.
Primitive data structure can be used to call the methods.	Non-primitive data structure cannot be used to call the methods.

B-tree	Binary tree
A node in b tree can have maximum 'M' number of child nodes where M is the order of the tree.	Unlike btree, binary tree can have maximum two children or two subtrees.
A btree is a sorted tree because its nodes are sorted in an inorder traversal.	A Binary tree is not a sorted tree A tree can be sorted either in inorder, preorder or postorder traversal.
The height of btree is $\log_M N$ where M is the order of tree and N is the number of nodes.	The height of binary tree is $\log_2 N$ where N is the number of nodes.
Btree is implemented when the data is stored in the disk.	Binary tree is implemented when the data is stored in RAM.
It is used in DBMS.	It is used in Huffman coding and code optimization.
Insertion of data or a key in a btree is more complex than the binary tree.	Insertion of data in a binary tree is easier as compared to btree.

Data type	Data structure
A Data type is one of the forms of a variable to which the value can be assigned of a given type only. This value can be used throughout the program.	A Data structure is a collection of data of different data types. This collection of data can be represented using an object and can be used throughout the program.
The implementation of data type is known as an abstract implementation.	The implementation of data structure is known as a concrete implementation.
It can hold value but not data. Therefore, we can say that it is data-less.	It can hold multiple types of data within a single object.
In case of data type, a value can be assigned directly to the variables.	In the case of data structure, some operations are used to assign the data to the data structure object.
There is no problem in the time complexity.	When we deal with a data structure object, time complexity plays an important role.
The examples of data type are int, float, char.	The examples of data structure are stack, queue, tree, graph.

BFS vs DFS

S. No.

	Parameters	BFS	DFS
1.	Stands for	BFS stands for Breadth First Search. BFS(Breadth First Search)	DFS stands for Depth First Search.
2.	Data Structure	uses Queue data structure for finding the shortest path.	DFS(Depth First Search) uses Stack data structure. DFS is also a traversal approach in which the traverse begins at the root node and proceeds through the nodes as far as possible until we reach the node with no unvisited nearby nodes.
3.	Definition	BFS is a traversal approach in which we first walk through all nodes on the same level before moving on to the next level. BFS can be used to find a single source shortest path in an unweighted graph because,	
4.	Technique		In DFS, we might traverse through more edges to reach a

		in BFS, we reach a vertex with a minimum number of edges from a source vertex.	destination vertex from a source.
5.	Conceptual Difference	BFS builds the tree level by level.	DFS builds the tree sub-tree by sub-tree.
6.	Approach used	It works on the concept of FIFO (First In First Out). BFS is more suitable for searching vertices closer to the given source.	It works on the concept of LIFO (Last In First Out). DFS is more suitable when there are solutions away from source.
7.	Suitable for		DFS is more suitable for game or puzzle problems. We make a decision, and then explore all paths through this decision. And if this decision leads to win situation, we stop.
8.	Suitable for Decision Treestheirwinning	BFS considers all neighbors first and therefore not suitable for decision-making trees used in games or puzzles.	The Time complexity of DFS is also $O(V + E)$ when Adjacency List is used and $O(V^2)$ when Adjacency Matrix is used, where V stands for vertices and E stands for edges.
9.	Time Complexity	The Time complexity of BFS is $O(V + E)$ when Adjacency List is used and $O(V^2)$ when Adjacency Matrix is used, where V stands for vertices and E stands for edges.	Here, children are visited before the siblings.
10.	Visiting of Siblings/ Children	Here, siblings are visited before the children.	The visited nodes are added to the stack and then removed when there are no more nodes to visit.
11.	Removal of Traversed Nodes	Nodes that are traversed several times are deleted from the queue.	DFS algorithm is a recursive algorithm that uses the idea of backtracking
12.	Backtracking	In BFS there is no concept of backtracking.	DFS is used in various applications such as acyclic graphs and topological order etc.
13.	Applications	BFS is used in various applications such as bipartite graphs, shortest paths, etc.	DFS requires less memory.
14.	Memory	BFS requires more memory.	DFS is not optimal for finding the shortest path.
15.	Optimality	BFS is optimal for finding the shortest path.	DFS has lesser space complexity because at a time it needs to store only a single
16.	Space complexity	In BFS, the space complexity is more critical as compared to time complexity.	

- | | | | |
|-----|--------------|--|--|
| | | | path from the root to the leaf node. |
| 17. | Speed | BFS is slow as compared to DFS. | DFS is fast as compared to BFS. |
| 18. | When to use? | When the target is close to the source, BFS performs better. | When the target is far from the source, DFS is preferable. |